



**Instituto Universitario de Ciencias de la Educación
Universidad de Salamanca**

Documentación del proyecto

**Web institucional del IUCE
Anexo C: Documentación Técnica**

Elaborado por
Enrique González Gutiérrez

Salamanca, septiembre de 2026

Índice

Documentación técnica de programación.....	3
C.1. Introducción.....	3
C.2. Estructura de directorios.....	3
C.3. Puesta en marcha, compilación y ejecución.....	4
C.3.1. Requisitos e instalación en local.....	4
C.3.2. Variables de entorno.....	4
C.3.3. Comandos.....	5
C.4. Manual del programador.....	5
C.4.1. Convenciones generales.....	5
C.4.2. Servicios principales (src/lib).....	6
C.4.3. Cómo añadir contenido editable a una página.....	6
C.4.4. Endpoints de la API.....	7
C.4.5. Scripts de migración y carga de datos.....	8
C.4.6. Correo transaccional.....	9
C.4.7. Internacionalización para el programador.....	9
C.5. Pruebas del sistema.....	9
C.6. Despliegue en producción.....	10

Anexo C

Documentación técnica de programación

C.1. Introducción

Este anexo documenta el sistema desde la perspectiva del desarrollo y está dirigido a quien tenga que mantener o evolucionar el código. Recoge la organización del proyecto, la puesta en marcha, el manual del programador (servicios, API, contenido editable, scripts de datos), la estrategia de pruebas y el procedimiento de despliegue.

El proyecto es una aplicación full-stack en un único repositorio, escrita íntegramente en TypeScript sobre Next.js 14 con el App Router: no hay un backend separado, sino rutas y componentes que conviven bajo `src/`, y un esquema de datos (`prisma/schema.prisma`) que actúa como fuente única de verdad de la que derivan tanto la sincronización con la base de datos como los tipos del resto de capas.

C.2. Estructura de directorios

```
iuce-web/
├── prisma/          schema.prisma (modelo) · seed.ts (semilla)
├── public/          logos, imágenes y documentos públicos
│   └── uploads/     imágenes subidas + histórico (fuera de git)
├── storage/intranet/ documentos internos (fuera de git y public/)
├── scripts/         migración y carga de datos (véase C.4.5)
│   └── data/        datos congelados de los scripts (json)
├── docs/           design/ (prototipos) · manuales/ (esta doc.)
└── src/
    ├── middleware.ts idioma /en + protección de zonas privadas
    ├── auth.config.ts sesión edge-safe (JWT, roles)
    └── app/
        ├── (public)/ sitio público + área de miembros /miembros
        ├── (admin)/backstage/ panel: news, pages, visualizacion,
        │               members, groups, events, projects, files,
        │               messages, intranet, settings
        ├── api/       REST: admin/, intranet/, contact, auth
        ├── auth/signin/ login del panel
        └── feed.xml/, sitemap.ts, robots.ts, opengraph-image...
    ├── components/   admin/ auth/ contact/ instituto/ intranet/
        │               investigacion/ layout/ news/ stats/ ui/
    └── lib/          servicios (C.4.2)
        └── content/  registros de fábrica ES+EN (bloques, listas,
            │          grupos, miembros, eventos, páginas ocultables)
```

Las pruebas unitarias viven junto al código que verifican, en carpetas `__tests__` (`src/lib/__tests__` y `src/components/__tests__`).

C.3. Puesta en marcha, compilación y ejecución

C.3.1. Requisitos e instalación en local

Se necesitan Node.js (18.17 o superior, requisito de Next.js 14), Docker con Docker Compose y Git. Docker se emplea únicamente para la base de datos en desarrollo; la aplicación corre con el servidor de desarrollo de Next.js:

```
git clone <URL-del-repositorio> iuce-web
cd iuce-web
npm install                # instala dependencias y genera el cliente
Prisma
cp .env.example .env       # rellenar variables (véase C.3.2)
docker compose up -d       # PostgreSQL 16 en el puerto 5433
npm run db:push            # crea el esquema en la base de datos
npm run db:seed            # contenido de fábrica + cuenta SUPER_ADMIN
npm run dev                # http://localhost:3000
```

El puerto de PostgreSQL es el 5433 para no chocar con la base de datos local de IUCE Reservas (5432). El panel queda en <http://localhost:3000/backstage>; la cuenta inicial la crea la semilla con las variables `ADMIN_EMAIL` / `ADMIN_PASSWORD` (por defecto, la cuenta institucional del Instituto con una contraseña de desarrollo que debe cambiarse en producción). Para reconstruir el contenido completo (histórico de noticias incluido) se ejecutan además los scripts de datos de C.4.5.

C.3.2. Variables de entorno

Variable	Uso
<code>DATABASE_URL</code>	Cadena de conexión de PostgreSQL.
<code>NEXTAUTH_URL</code>	URL pública de la aplicación (localhost en desarrollo; dominio real en producción).
<code>NEXTAUTH_SECRET</code>	Secreto de firma de las sesiones JWT (generar uno propio por entorno).
<code>RESEND_API_KEY</code>	Clave del servicio de correo. Sin ella no salen correos: la solicitud de acceso al área de miembros devuelve el enlace en modo desarrollo y el contacto informa del fallo.
<code>EMAIL_FROM</code>	Remitente de los correos (debe pertenecer a un dominio verificado en Resend).
<code>CONTACT_TO</code>	Buzón que recibe los avisos del formulario de contacto.
<code>DEEPL_API_KEY</code>	Opcional. Con clave, al guardar noticias y bloques se genera su versión inglesa; sin clave, la traducción se omite.
<code>ADMIN_EMAIL</code> / <code>ADMIN_PASSWORD</code>	Credenciales de la cuenta <code>SUPER_ADMIN</code> que crea la semilla.
<code>MIGRATION_EXPORT_DIR</code>	Solo para la migración: carpeta del export de WordPress.

Tabla 1. *Variables de entorno.*

C.3.3. Comandos

Comando	Acción
<code>npm run dev</code>	Servidor de desarrollo con recarga en caliente.
<code>npm run build / npm run start</code>	Compilación de producción y arranque del servidor compilado.
<code>npm run lint</code>	ESLint con la configuración de Next.
<code>npm run test / npm run test:watch</code>	Suite de pruebas Vitest (una pasada / en vigilancia).
<code>npm run db:push</code>	Sincroniza el esquema Prisma con la base de datos.
<code>npm run db:seed</code>	Ejecuta la semilla (idempotente).
<code>npm run db:generate</code>	Regenera el cliente Prisma.

Tabla 2. *Comandos npm del proyecto.*

Avisos prácticos (Windows/desarrollo): tras un `npm run build` de verificación conviene borrar la carpeta `.next` antes de volver a `npm run dev` (arrancar sobre artefactos de producción produce estados incoherentes); tras un `db:push` con modelos nuevos hay que reiniciar el servidor de desarrollo (y, si persisten tipos antiguos, limpiar `.next`); y `prisma generate` falla con `EPERM` si el servidor está corriendo, porque la DLL del cliente está bloqueada: parar el servidor, generar y arrancar de nuevo.

C.4. Manual del programador

C.4.1. Convenciones generales

- TypeScript en modo estricto en todo el proyecto.
- Server Components por defecto; los componentes de cliente ("use client") se reservan para interactividad y reciben el idioma y los datos como propiedades (nunca importan helpers de servidor).
- La lógica de negocio y el acceso a datos viven en `src/lib` (servicios), no en las páginas; las páginas orquestan y presentan.
- Toda entrada de usuario se valida en servidor con Zod; los esquemas del panel están centralizados en `lib/admin-schemas.ts`.
- Las rutas de administración comprueban la sesión con el guard común (`lib/admin-guard.ts`) además de la protección del middleware.
- La interfaz del panel está íntegramente en español; los textos públicos bilingües siguen el sistema de contenido de C.4.4.

C.4.2. Servicios principales (src/lib)

Módulo	Responsabilidad
news-service.ts	Consultas de noticias públicas e internas (listados, filtros, detalle, RSS) con resolución de idioma.
content-blocks-service.ts	getBlock/getListBlock: resolución en cascada BD → registro EN → registro ES de bloques y listas.
projects-service.ts	Proyectos públicos (activos y del IUCE) y su explorador.
page-visibility.ts	assertVisible (404 si la página está oculta, salvo sesión de administración) y getHiddenPaths para menús y sitemap.
intranet-access.ts	Regla de acceso al área de miembros (veto, lista de autorizados, correo de miembro con alta automática).
intranet-session.ts	Utilidades de la sesión de miembro (identidad, avatar con la foto de su ficha).
auth.ts · auth.config.ts	NextAuth v5: proveedor de credenciales del panel (bcrypt) y proveedor «intranet» (magic link) con rol INTRANET; configuración edge-safe para el middleware.
email.ts	Plantillas HTML institucionales (enlace de acceso, aviso y acuse de contacto) con logo incrustado (adjunto en línea) y versión de texto plano.
translate.ts	Cliente de DeepL para la traducción al guardar y bajo demanda (se omite sin clave).
orcid.ts	Últimos artículos de la dirección vía la API pública de ORCID (caché de 24 h; null si falla, y la página usa la lista editable de reserva).
rate-limit.ts	Limitación de tasa en memoria para los puntos sensibles.
metadata.ts	metadataBilingue: títulos y descripciones de página en el idioma de la petición.
locale.ts · locale-server.ts	Idioma en cliente (pathLocale, withLocale, pick) y en servidor (getLocale desde la cabecera x-locale).
validations.ts · admin-schemas.ts	Esquemas Zod compartidos (contacto, entidades del panel).
slugify.ts	Generación de slugs estables (con normalización de tildes y ñ).
prisma.ts	Cliente Prisma único (singleton) del proceso.
icon-map.ts	Catálogo de iconos seleccionables en las listas estructuradas.
content/*	Registros de fábrica: page-blocks y list-blocks (ES y EN), groups, members, events, news (categorías) y public-pages (páginas ocultables).

Tabla 3. Servicios y módulos de src/lib.

C.4.3. Cómo añadir contenido editable a una página

Para incorporar un texto o una lista nuevos al sistema de edición: (1) declarar la pieza en el registro correspondiente (lib/content/page-blocks.ts o list-blocks.ts) con su página, clave, etiqueta y valor de fábrica; (2) añadir su traducción de fábrica al registro -en homólogo; (3)

leerla en la página con `getBlock(pageSlug, blockKey)` o `getListBlock(...)`. La pieza aparece automáticamente en el panel (Contenido → Páginas) con su editor y su botón de restablecer. Dos reglas: las listas no se traducen automáticamente (sus versiones inglesas se mantienen en el registro -en o desde el panel) y las claves nunca se renombran a la ligera (la fila editada en base de datos quedaría huérfana).

C.4.4. Endpoints de la API

Todos los endpoints devuelven JSON y validan con Zod. Los de `/api/admin` exigen sesión de administración (middleware + guard); los del área de miembros, la sesión que se indica.

Endpoint	Métodos	Función
<code>/api/contact</code>	POST	Envío del formulario de contacto (valida, guarda, notifica y acusa; con límite de tasa).
<code>/api/intranet/request-link</code>	POST	Solicitud del enlace de acceso al área de miembros (regla de acceso + límite de tasa).
<code>/api/intranet/files/[id]</code>	GET	Descarga de un documento interno (requiere sesión INTRANET o de administración).
<code>/api/intranet/profile</code>	PUT	Actualización de la ficha del propio miembro.
<code>/api/intranet/profile/photo</code>	POST	Subida de la foto del propio miembro (sharp, 512 px).
<code>/api/intranet/dev-access</code>	GET	Acceso directo de desarrollo (devuelve 404 en producción).
<code>/api/auth/[...nextauth]</code>	—	Rutas internas de NextAuth (login del panel y callback del magic link).

Tabla 4. Endpoints públicos y del área de miembros.

Recurso	Métodos	Función
<code>news · news/[id]</code>	GET POST · GET PUT DELETE	CRUD de noticias.
<code>members · members/[id]</code>	GET POST · PUT DELETE	CRUD de miembros.
<code>members/photo · members/photos</code>	POST · GET	Subida de foto (sharp 512 px) y biblioteca de fotos existentes.
<code>groups · groups/[id]</code>	GET POST · PUT DELETE	CRUD de grupos de investigación.
<code>events · events/[id]</code>	GET POST · PUT DELETE	CRUD de eventos.
<code>projects · projects/[id]</code>	GET POST · PUT DELETE	CRUD de proyectos (incluida la marca «del IUCE»).
<code>files · files/[id]</code>	GET POST · DELETE	Biblioteca de archivos (subidas a /uploads).
<code>messages/[id]</code>	PUT DELETE	Estado y borrado de mensajes de contacto.

Recurso	Métodos	Función
content-blocks	GET PUT	Lectura y guardado de bloques y listas editables.
page-visibility	GET PUT	Interruptores de visibilidad de páginas.
intranet/users · intranet/users/[id]	GET POST · PUT DELETE	Usuarios del área de miembros (autorizar, vetar).
intranet/files · intranet/files/[id]	GET POST · PUT DELETE	Documentos internos (metadatos y fichero).
accounts	GET POST	Cuentas de administración (solo SUPER_ADMIN).
translate	POST	Traducción bajo demanda vía DeepL.

Tabla 5. Endpoints de administración (/api/admin).

C.4.5. Scripts de migración y carga de datos

Los scripts de scripts/ reconstruyen el contenido real. Todos son idempotentes y los que tocan datos editables son de solo-relleno: no pisan cambios hechos desde el panel. En un despliegue desde cero se ejecutan tras la semilla, en el orden de la tabla.

Script	Función
migrate-wordpress.ts	Migración del export de WordPress: 212 noticias (2010–2026) con sus imágenes, categorías mapeadas y fichas de miembros con foto y correo institucional.
recover-external-images.js	Recupera las imágenes de noticias antiguas que enlazaban a servicios externos desaparecidos (Flickr, Wayback Machine).
copy-audit-media.js	Copia los medios recuperados en la auditoría de contenido de la web antigua (documentos, planos, PDF).
import-projects.ts	Importa los proyectos de la Tabla 4 de la memoria de acreditación (scripts/data/projects.json).
assign-member-groups.ts	Asigna grupo y responsables a los miembros según la evidencia recopilada (scripts/data/member-groups.json) y aplica las bajas dictadas por la dirección.
patch-member-contacts.ts	Completa extensiones telefónicas y ORCID verificados de la dirección y el personal.
apply-news-en.ts	Aplica los lotes de traducción de noticias (scripts/data/news-en/batch-*.json); solo-relleno, con --force para regenerar.
import-late-news.ts	Noticias publicadas en la web antigua DESPUÉS del export congelado (scripts/data/late-news/*.json, con ES y EN); solo-relleno, con --force para pisar. Mientras la web antigua siga viva, las noticias nuevas se añaden aquí.
backfill-news-en.ts	Vía alternativa: traduce con DeepL las noticias sin versión inglesa (requiere clave).
fetch-orcids.py	Auxiliar de verificación de ORCID contra la API pública.

Tabla 6. *Scripts de datos.*

C.4.6. Correo transaccional

lib/email.ts construye los tres correos del sistema (enlace de acceso, aviso de contacto con responder-a al remitente y acuse de recibo) con una plantilla común: cabecera institucional con el logo **incrustado como adjunto en línea** (de modo que se ve en cualquier cliente, sin depender de una URL pública), cuerpo con botón de acción, pie institucional y versión de texto plano. El SDK de Resend no lanza excepciones en los errores de API: los devuelve en el campo error de la respuesta, y las rutas lo comprueban explícitamente para no dar confirmaciones en falso.

C.4.7. Internacionalización para el programador

- El middleware reescribe /en/* a la ruta española con la cabecera x-locale; getLocale() (solo servidor) la lee. Los componentes de cliente reciben locale como propiedad.
- Todo enlace interno de una página bilingüe se construye con withLocale(href, locale) para no salirse del idioma.
- Los metadatos de página usan metadataBilingue(es, en); el html lang es dinámico.
- El conmutador de idioma es un <a> nativo a propósito (navegación de documento completo): con un Link de cliente, ambos idiomas compartirían la caché del router y el contenido no cambiaría. Cualquier enlace futuro que cruce idiomas debe mantener esa regla.
- Textos nuevos: en páginas con pocos literales se usa un diccionario inline const T = { es, en } y pick(locale, T); el contenido editable sigue el sistema de registros de C.4.3.

C.5. Pruebas del sistema

La suite de pruebas unitarias corre con Vitest (entorno jsdom para los componentes) y cubre la lógica pura más sensible y los componentes de interfaz básicos. A fecha de esta documentación la suite comprende **27 pruebas en 4 ficheros**, todas en verde: slugify (6, incluida la normalización de tildes y ñ), validaciones Zod (10), tratamiento de la imagen de portada en el contenido (5) y componentes de UI (6). Se ejecuta con `npm run test` y en modo vigilancia con `npm run test:watch`; las pruebas viven junto al código en carpetas `__tests__`.

La verificación funcional se completa con la compilación estricta de TypeScript y ESLint, y con la comprobación manual guiada de los flujos de usuario (incluido el correo real de extremo a extremo y la navegación de idioma, que debe verificarse con clic real en el navegador: las cargas directas de URL no reproducen el matiz de caché descrito en C.4.7).

C.6. Despliegue en producción

El modelo de producción replica el de IUCE Reservas en el CPD de la USAL: aplicación y PostgreSQL como contenedores Docker tras Apache 2 (proxy inverso con certificado Let's Encrypt). Lista de comprobación del despliegue:

- Variables de entorno de producción: NEXTAUTH_URL con el dominio real, NEXTAUTH_SECRET propio, credenciales de base de datos, RESEND_API_KEY y EMAIL_FROM de un dominio verificado.
- Cambiar la contraseña de la cuenta de administración inicial y dar de alta las cuentas reales (Configuración, con la cuenta SUPER_ADMIN).
- npm run build y arranque del contenedor de la aplicación; base de datos con db:push + seed + scripts de datos (C.4.5) o restauración de un volcado.
- Copiar los datos que no viajan con git: public/uploads (imágenes migradas y subidas), storage/ (documentos internos) y el volcado de la base de datos. Estos tres conjuntos forman también la copia de seguridad periódica.
- Verificar tras el arranque: envío real de un enlace de acceso y del formulario de contacto, navegación /en, robots y sitemap con el dominio real y redirecciones de la web antigua.
- Opcional: solicitar al CPD los registros DNS para un remitente de correo propio de la web.